MULTI-AGENT REPOSITORY INTELLIGENCE AND RISK ANALYSIS

Skopeo: design of a multi-agent repository risk analyst that proves what it finds

Thirteen agents investigate a GitHub repository together: an orchestrator plans and re-plans, eight specialists gather evidence in parallel, and four coordination agents correlate, challenge, score and fix. They communicate through a shared Evidence Store (a blackboard) and a persisted message log. Every claim cites its evidence, every finding is cross-examined by a red team, and nothing reaches GitHub without a human.

Team	Kushagra Goyal (`mardromus`) Mayank Hete (`Rancidcake`) Abhilaksh Saini
Institute	Department of Computer Science and Engineering, Symbiosis Institute of Technology, Pune
Repository	https://github.com/mardromus/Skopeo
Pattern	Hierarchical orchestrator–worker over a blackboard, with adaptive routing and an adversarial verifier
Stack	LangGraph StateGraph (Python 3.12+), FastAPI, React · LLM: Groq (`openai/gpt-oss-120b` , `openai/gpt-oss-20b`) · PostgreSQL or SQLite

13 + 1
agents, plus a question router

8
specialists running at once

233
events in the submitted trace

221
automated tests (SQLite and PostgreSQL)

01 Architecture and data flow
02 Agents, roles and responsibilities
03 Orchestration pattern and messages
04 Evidence of agentic behaviour
05 Ask the agents
06 Human control and guardrails
07 Upstream watches and notifications
08 Input and output
09 Technology choices
10 Robustness and safety
11 Deployment
12 Engineering practice and limitations

01 Architecture and data flow

01	02	03	04	05	06
Intake	**Plan**	**Investigate**	**Correlate**	**Red team**	**Score & act**
Validate the URL, clone at a commit, record the guardrails.	Profile the repository; choose agents, with a reason for each.	Eight specialists run in parallel and write evidence.	Link findings into compound risks and contradictions.	Challenge every finding; verify, reject or ask for more.	Rank P0–P4; propose fixes that wait for approval.

The orchestrator reviews the blackboard after every phase, so steps 3–5 can repeat: follow-ups, retries and benchmarks are scheduled when the evidence calls for them.

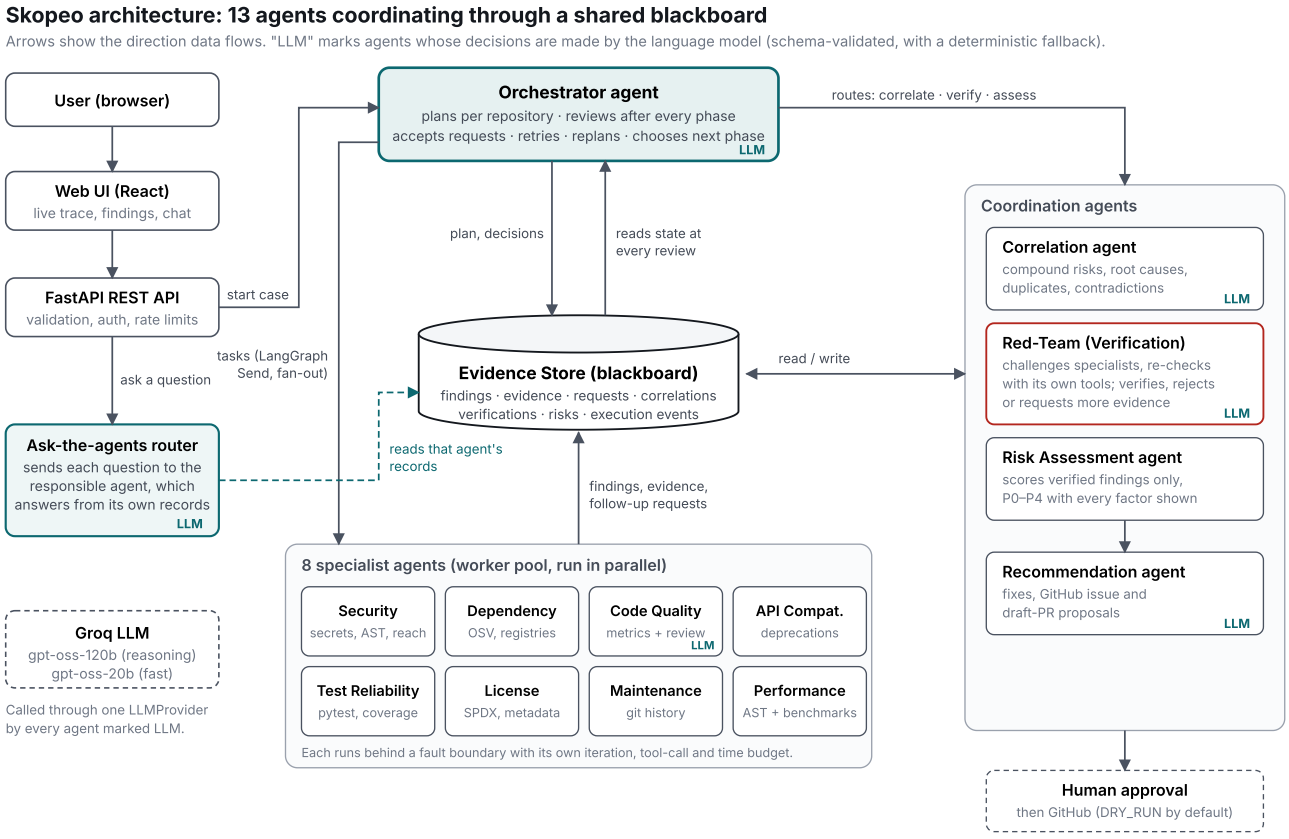


Figure 1. Every agent and every connection. Specialists never call each other: they write findings, evidence and follow-up requests to the Evidence Store, which the orchestrator reads at every review and the coordination agents read and write.

How a request flows. The user submits a GitHub repository (or picks one after signing in with GitHub) and chooses the case's guardrails: dry run, whether approved actions may reach GitHub, and whether repository code may execute. The API validates the request and opens a case. The **Orchestrator** profiles the repository and plans which specialists to run and why (it records a reason for every agent selected or skipped). LangGraph fans the tasks out with `Send`, so the specialists run in parallel; each posts findings, the evidence behind them, and requests for follow-up work to the Evidence Store. After every phase the orchestrator **reviews** the blackboard and decides the next step: dispatch more work (follow-ups, retries, benchmarks), correlate, verify, assess or finish. The **Correlation** agent links findings across specialists; the **Red-Team** agent challenges each finding and either verifies it, rejects it, or sends it back for more evidence; the **Risk** agent scores only verified findings; the **Recommendation** agent writes fixes and proposes GitHub actions that wait for human approval. After the case, the **Ask-the-agents router** sends each user question to the agent that owns the topic.

02 Agents, roles and responsibilities

Agent	Responsibility	Tools it uses	Decisions it makes
Orchestrator	Profiles the repository, plans per repository, reviews after every phase, accepts or declines other agents' requests, schedules follow-ups and retries, re-plans, routes the graph, enforces budgets and the deadline.	Repository profile, blackboard queries, budgets	`orchestrator.plan`, `orchestrator.review` (LLM, reasoning model)
Security	Committed secrets, insecure code patterns, prompt-injection text, whether vulnerable packages are used and reachable.	Secret patterns, Python AST, import graph; Semgrep and Dependabot when available	Own plan; extends it on discoveries
Dependency Health	Vulnerabilities, freshness, abandonment, pinning and conflicts (PyPI, npm, Go, crates.io, Maven).	Manifest parsers, OSV.dev, PyPI/npm registries (offline snapshot for demos)	Raises impact-analysis requests
Code Quality	Complexity, nesting, size, swallowed exceptions, duplication, data-access maintainability.	AST metrics	`code_quality.review` (LLM) of the most complex functions
API Compatibility	Public API, deprecations still in use, breaking changes in history, whether a dependency upgrade is drop-in.	AST, git history, migration rules with changelog links	Upgrade-compatibility verdicts
Test Reliability	Test discovery, assertion quality, flakiness signals, which code is untested, real test runs.	pytest + coverage.py (sandboxed), AST	Coverage mapping on request
License Compliance	Project license vs metadata, copyleft files, dependency license compatibility.	License fingerprints, SPDX headers, registry metadata	—
Maintenance Activity	Commit cadence, contributor concentration (bus factor), stale modules, releases, PR activity.	git log, GitHub REST (with a token)	—
Performance	Static suspects (N+1 queries, network calls in loops) versus measured problems.	AST, benchmark runner	Runs benchmarks the red team asks for
Correlation	Builds a typed link graph across all findings: compound risk, shared root cause, causal chain, dependency link, duplicate, contradiction. Sends contradictions to the red team.	Link graph over the blackboard	`correlation.assess` (LLM) accepts or rejects each candidate
Red-Team (Verification)	Distrusts the other agents. Re-hashes cited lines, rebuilds the import graph, re-runs failing tests, checks placeholders, compares severity with advisory data, demands measurement for performance claims.	Its own copies of the tools above	`verification.judge` (LLM, reasoning model): verified / rejected / needs more evidence

Risk Assessment	Scores verified findings and compound risks P0–P4 with every factor stored: severity × confidence × impact × exploitability × correlation multiplier.	Transparent formula	Deterministic by design (auditable)
Recommendation	Turns risks into fixes, GitHub issues and draft-PR proposals that need human approval.	Templates, approval gate	`recommendation.write` (LLM)
Ask-the-agents router	Routes each user question to the agent that owns the topic; that agent answers only from its own findings, evidence and code excerpts, and cites them.	Keyword scoring, blackboard, cloned repository	`chat_route` (LLM, fast) and `chat_answer` (LLM, reasoning)

Code: `backend/app/agents/` (one module per agent; `registry.py` is the only place names map to classes), `backend/app/chat/service.py` (router). Full specifications: `docs/agent-specifications.md`.

03 Orchestration pattern and messages

Named pattern: hierarchical orchestrator–worker over a blackboard. The orchestrator issues `AgentTask` s (agent, objective, mode, strategy, focus, trigger, attempt, round); workers return `TaskResult` s. Workers do not call each other: they publish to the Evidence Store, and later work is driven by what is on the board. That is how one agent's discovery changes another agent's task.

The investigation graph: the orchestrator's review chooses every next step

LangGraph StateGraph in backend/app/graph/builder.py. Only "review" has several exits; which one is taken depends on what the blackboard shows.

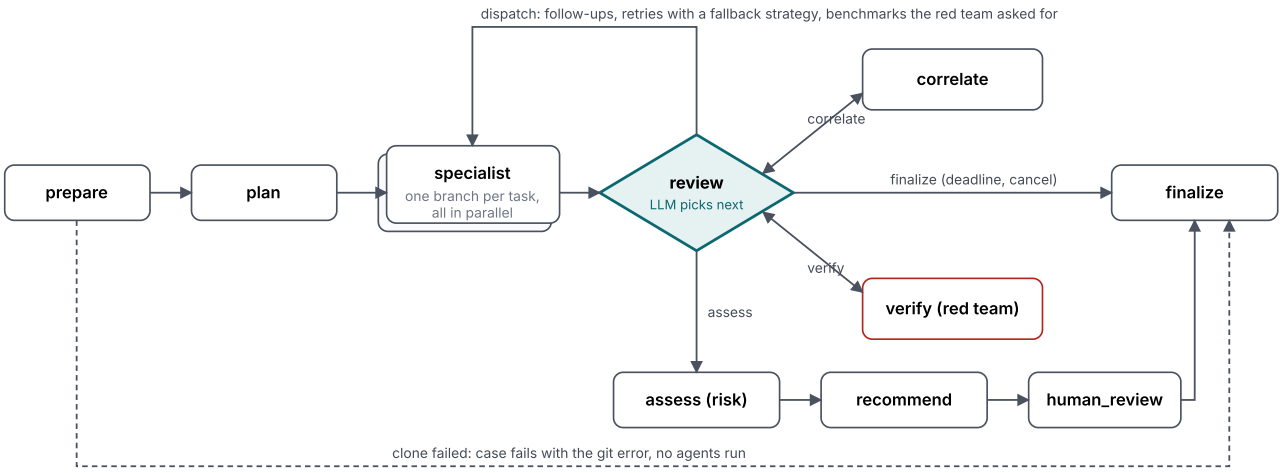


Figure 2. The LangGraph state machine. `review` runs after every phase and returns one of dispatch · correlate · verify · assess · finalize.

WHY ORCHESTRATOR–WORKER

The work needed is only known after the first round: a vulnerable library in the login path calls for three new tasks, a crashed agent for a degraded retry, an unmeasured performance claim for a benchmark. A fixed pipeline cannot express this.

WHY A BLACKBOARD

Thirteen agents passing results directly would need 156 pairwise channels. One typed store gives every agent the same evidence, makes every write auditable, and let a later agent (the chat router) reuse all of it.

WHY AN ADVERSARIAL VERIFIER

The specialists' tools produce false positives. A separate agent whose only job is to disprove findings removes them before scoring.

WHY PARALLEL FAN-OUT

The specialists are independent; LangGraph `Send` runs eight at once, and the next review starts when the whole superstep completes.

How agents communicate

Message	From → to	Effect
`investigation_requested`	any agent → orchestrator	asks for more work (impact analysis, usage analysis, upgrade compatibility, coverage, benchmark)
`request_decided` , `replan`	orchestrator → requester, all	request accepted (with assigned tasks) or declined (with reason); plan extended
`agent_failed` , `agent_retry_scheduled`	agent → orchestrator → agent	failure isolated; retried with a fallback strategy or reported unavailable
`challenge`	red team → specialist	a specific question put to the agent that made a finding
`finding_verified` , `finding_rejected` , `verification_completed`	red team → specialist	ruling; "needs more evidence" also becomes a request
`correlation_created` , `correlation_updated`	correlation ↔ red team	relationships found, re-assessed and ruled on
`orchestrator_decision`	orchestrator → all	review outcome and the next phase, with its reasons
`risk_assessed` , `human_approval_required`	risk → recommendation → human	priorities, then proposals held for approval

Every message is a persisted `execution_events` row (sender, receiver, type, message, payload, timestamp, sequence number). The UI's Execution trace tab and the submitted trace read this table; nothing is written for display only.

04 Evidence of agentic behaviour

Numbers in brackets are event sequence numbers in the submitted execution trace (reference repository, 15 Oct 2026).

Behaviour	What happened in the run
Planning	`[004]` The orchestrator plans 8 specialists in parallel from the repository profile; each specialist posts its own plan (`[006]`, `[008]`, …) and the Code Quality agent extends its plan after its first tool result `[035]`.
Tool use	Real tools run: the test suite executes (1 failed, 6 passed) `[054]`; a benchmark is executed `[167]` and measures 51 → 2001 queries `[168]`.
Failure and recovery	The performance agent fails (controlled fault) `[040]`; the orchestrator retries it with a `static_only` strategy `[060]` instead of stopping; the case ends `completed_with_limitations` with that limitation stated.

Agents changing each other's work	The dependency agent asks for an impact analysis of PyJWT 1.7.1 `[016]`; the orchestrator accepts it and re-plans with three new tasks for three other agents `[061]`–`[062]`, and records why no performance follow-up was needed.
Conflict resolution	Performance and Code Quality disagree about `list_orders_with_items()`; the correlation agent flags a contradiction and hands it to the red team `[087]`–`[088]`; the red team demands a measurement `[134]`; the orchestrator schedules a benchmark `[156]`–`[159]`; with the measurement, the red team verifies the N+1 finding `[180]` and rejects the "acceptable" assessment `[182]`.
Critique	The red team rejects two AWS keys as documented example values in test fixtures `[100]`, `[102]`, and, challenged on reachability `[097]`, verifies the vulnerable dependency through its own import graph.
Cross-agent reasoning	Four agents' findings become one compound risk, "vulnerable PyJWT in authentication code with insufficient tests", ×1.75, scored P0 (94.5) `[090]`, `[152]`.
Memory	Graph state (round, pending tasks, reviewed ids, completed-task keys for de-duplication, blackboard version at last correlation/verification) plus the persistent blackboard; a later round re-correlates only because new evidence arrived.

> **Where the LLM decides, and how it is kept safe.** Agents never call a model directly. They call `LLMProvider.decide()` with a Pydantic output schema, trusted structured state, repository text marked as untrusted, and a deterministic policy. The model's answer must validate against the schema and the agent's guardrails (the orchestrator may only route to the five allowed steps; the red team may not reject a finding unless one of its checks failed; the chat may cite only finding ids that exist). If the model fails, times out, returns invalid output or the per-case call budget is spent, the policy decides and the trace records `policy-fallback`. Every decision records who made it (`llm:<model>` or `policy`). The submitted trace was captured without a model key, so its decisions show `policy`; the deployed site runs the same graph with Groq.

05 Ask the agents

After an investigation, a user can ask questions in the case's *Ask the agents* tab. The router scores the question against each agent's domain (and, with Groq, asks the fast model to choose, validated against the list of real agents). The chosen agent then answers **only** from what it recorded: its findings, their evidence and verification, and code excerpts from the clone. Every cited finding id is checked against the case before the answer is shown.

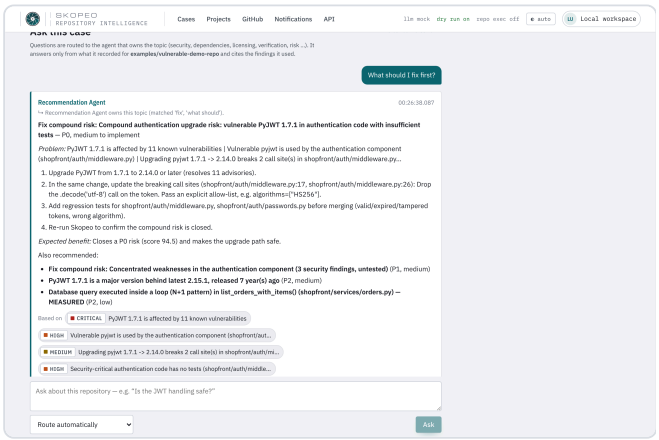


Figure 3. A question routed to the Recommendation agent, with the routing reason and the findings it relied on.

In the submitted trace, "What should I fix first?" went to the Recommendation agent, "Is the JWT handling safe?" to the Security agent, "Which findings did the red team reject?" to the Verification agent and a license question to the License agent, each answer citing real findings. Conversations are stored per user and per case.

06 Human control and guardrails

Skopeo can act on GitHub and can, if allowed, run a repository's own tests. Both are off by default and both stay under the user's control at three levels.

DRY RUN · THREE LEVELS

A personal default saved on the account (the *dry run* switch in the header); a choice in the pop-up shown before every case starts, from the home page or a GitHub import; and a switch on the case page that can be flipped while the case runs, which decides what the next approval does.

NAMED HUMAN APPROVAL

Recommended fixes become proposed GitHub issues and draft pull requests. Each needs a named approval; with dry run on the approval is recorded and nothing leaves the server. Skopeo never merges, deploys, deletes files or rotates credentials.

CODE EXECUTION

Off by default: agents read code but never run it. Changing the setting asks for confirmation twice, and the server refuses it unless `SKOPEO_ALLOW_CASE_EXECUTION` is set, so a shared deployment cannot be made to run untrusted code.

PRIVATE REPOSITORIES

Imported with the owner's GitHub token, which is stored encrypted (Fernet). Private cases are visible only to their owner; others receive 404.

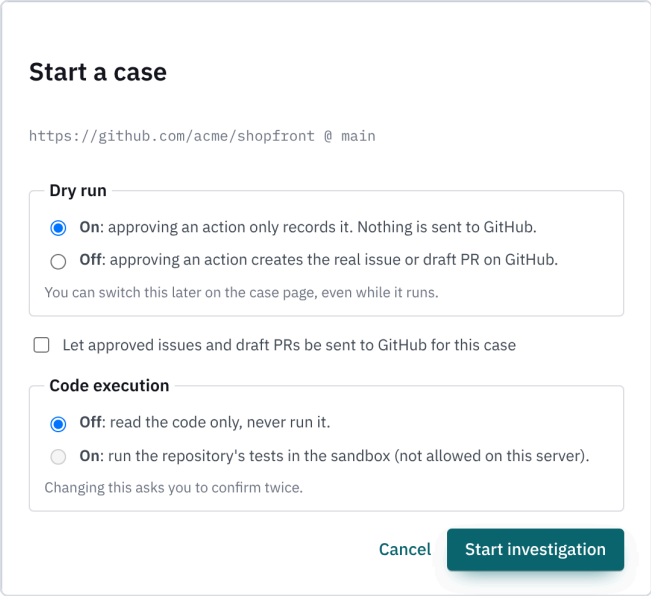


Figure 4. The pop-up shown before every case: dry run, GitHub actions and code execution for this case.

07 Upstream watches and notifications

A **project** is the user's own code: a GitHub repository or uploaded source files and manifests. A **watch** follows an open-source library that project depends on. On each check (hourly, daily, weekly or on demand) Skopeo diffs the library's public API between the version the project pins and the new release (removed and moved symbols, changed signatures, changed behaviour and contracts, additions) and maps every change to the call sites in the project.

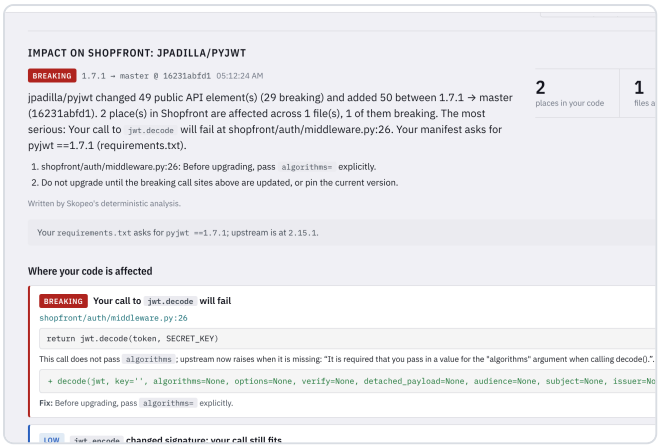


Figure 5. Watching `jpadilla/pyjwt` : 49 public API elements changed between 1.7.1 and 2.15.1 (29 breaking); the project's call to `jwt.decode` at `shopfront/auth/middleware.py:26` will fail because `algorithms` is now required.

Channel	Delivery
Email	SMTP, or HTTPS APIs (Brevo, Resend) on hosts that block SMTP ports; addresses are verified with a code
Slack, Discord	Incoming webhooks with formatted messages
Webhook	JSON payload signed with HMAC-SHA256 (`X-Skopeo-Signature`), public destinations only

Rules choose which events reach which channel: case finished or failed, high-priority risk found, an upstream change that affects the project, a failed watch check. Every delivery is logged with its status.

08 Input and output

Input (`POST /api/investigations` , schema `InvestigationCreate`): `repository_url` (https GitHub URL only), `branch` (validated ref, default `main`), `analysis_depth` (`quick | standard | deep`), `enable_github_actions` (bool), and optional per-case `dry_run` and `sandbox_execution` (null keeps the user's or server's default). Unknown fields are rejected. Related endpoints: questions `POST /api/investigations/{id}/chat` ; switching dry run `PATCH /api/investigations/{id}/options` ; personal defaults `PUT /api/auth/preferences` ; repository import `POST /api/github/import` .

Output (`GET /api/investigations/{id}/report` , also as Markdown): status, overall risk, findings (with evidence ids, confidence factors and status), the red team's verifications, correlations, risks P0–P4, recommendations, action proposals, agent requests, a per-agent coverage table and the full event trace. Full schemas and real samples are in the submitted Input and Output Definitions.

```
{ "repository_url": "https://github.com/pallets/itsdangerous", "branch": "main",
  "analysis_depth": "standard", "enable_github_actions": false, "dry_run": true }
→ 202 { "investigation_id": "…", "status": "queued", "options": { "dry_run": true,
"sandbox_execution": null }, … }
→ report: { "status": "completed_with_limitations", "overall_risk": {"score": 97.8, "level":
"critical"},
            "findings": [ … 24 … ], "verifications": [ … ], "correlations": [ … ], "risks": [ … ],
"events": [ … 233 … ] }
            (report figures from the reference run in the trace)
```

09 Technology choices

Choice	Why
LangGraph	An explicit state machine with conditional edges (the orchestrator's routing), `Send` fan-out/fan-in for parallel specialists, reducers to merge parallel results, and recursion limits. The control flow is decided at run time, which a chain or a no-code workflow tool cannot do.

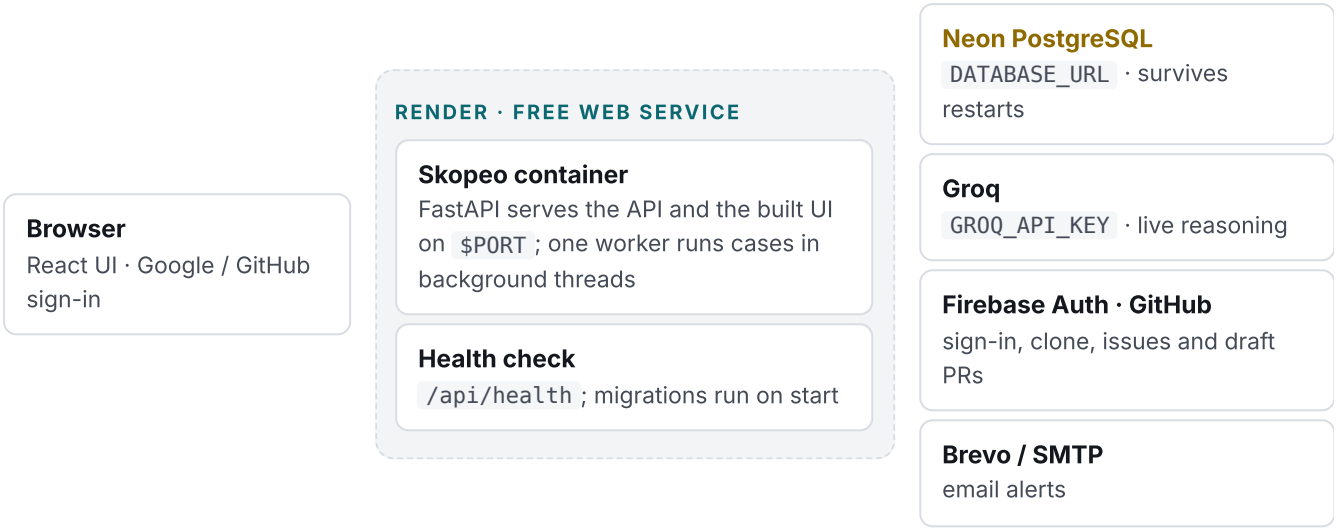
Groq, `openai/gpt-oss-120b` for reasoning	Planning, review/routing, red-team judgements and chat answers need careful reasoning; Groq serves this open-weight model fast enough to keep a live case interactive, with JSON-mode structured output.
Groq, `openai/gpt-oss-20b` for fast tasks	Correlation assessment, code review summaries, recommendation wording and question routing are short, frequent calls where latency and rate limits matter more.
Provider abstraction + deterministic policies	One interface for Groq, OpenAI, any OpenAI-compatible server, or no model at all; agents keep working (and stay testable) when the model is down or rate-limited.
FastAPI + Pydantic 2	Typed schemas end to end; the same models validate API input and every LLM output.
SQLAlchemy 2 + Alembic; SQLite or PostgreSQL	A real relational blackboard with migrations. SQLite needs no setup locally; PostgreSQL (free Neon) keeps data on hosts without a persistent disk.
Deterministic tools first (AST, OSV, pytest, coverage, git)	Evidence comes from tools; the LLM judges and explains it, which keeps findings reproducible and checkable by the red team.
React + TypeScript + Vite	A typed client mirroring the API: live trace, agent timeline, findings, rulings, chat, projects and notifications, in light and dark themes.
Firebase Authentication	Google and GitHub sign-in without storing passwords; the API verifies ID tokens itself, with no service account.
Docker, Render, Neon	One container serves UI and API; free hosting with a free PostgreSQL database so data survives restarts.

10 Robustness and safety

Situation	Behaviour
Invalid input	Rejected with HTTP 422 before any agent runs (non-GitHub URL, bad branch, unknown field, invalid depth).
A specialist crashes or times out	Only that run fails; evidence already written is kept; siblings continue; retried once with a fallback strategy, then reported unavailable in the coverage table.
LLM unavailable, invalid or over budget	The deterministic policy decides and the trace says so.
A claim cannot be settled	It stays "needs more evidence" and is excluded from the risk score.
Runaway investigation	Round limits, per-agent iteration, tool-call and time budgets, a deadline guard that routes to assessment, a hard timeout; users can cancel.
Hostile repository content	Treated as data, never instructions; no shell, allowlisted argv commands with limits and a stripped environment; untrusted code is not executed unless execution is allowed and chosen; secrets redacted everywhere.
Outbound actions	GitHub actions need a named approval and respect dry run; webhooks are restricted to public destinations; credentials and tokens never appear in logs or error messages.
Abuse of a public deployment	Optional access key for scans, per-client rate limits, a strict Content-Security-Policy, owner-only private cases.

Server restart	Cases left running are marked interrupted; with PostgreSQL, accounts, cases, projects and watches persist.

11 Deployment



Everything is configured through environment variables (`render.yaml` declares them); the step-by-step guide is `docs/deployment.md` . Without a model key the same container runs with deterministic policies.

12 Engineering practice and limitations

STRUCTURE

`backend/app/{agents, graph, llm, chat, services, tools, security, api, models, schemas, watch, notifications}` , `frontend/src` , `examples/` (reference repository), `docs/` .

TESTS

221 automated tests (unit, integration, end-to-end API): agents, the full multi-agent run, fault isolation, verification, prompt injection, GitHub import, chat routing, case options, email delivery and the database. They pass on SQLite and PostgreSQL.

HYGIENE

No secrets in the repository (keys come from the host), Ruff and ESLint clean, TypeScript strict, an Alembic migration for every schema change.

Limitations

- Deep static analysis is Python-first; other languages get manifest, advisory, license and pattern coverage.
- Test execution of real repositories requires execution to be allowed and chosen; otherwise the test agent is static.
- Reachability is import-graph based; dynamic imports can be missed (reported as inconclusive).
- Without a model key the agents' judgement comes from transparent rule-based policies; the event flow and state transitions are the same.
- The free Render plan sleeps after 15 minutes without traffic, so the first request after a pause waits about a minute.

A Screenshots of the running system

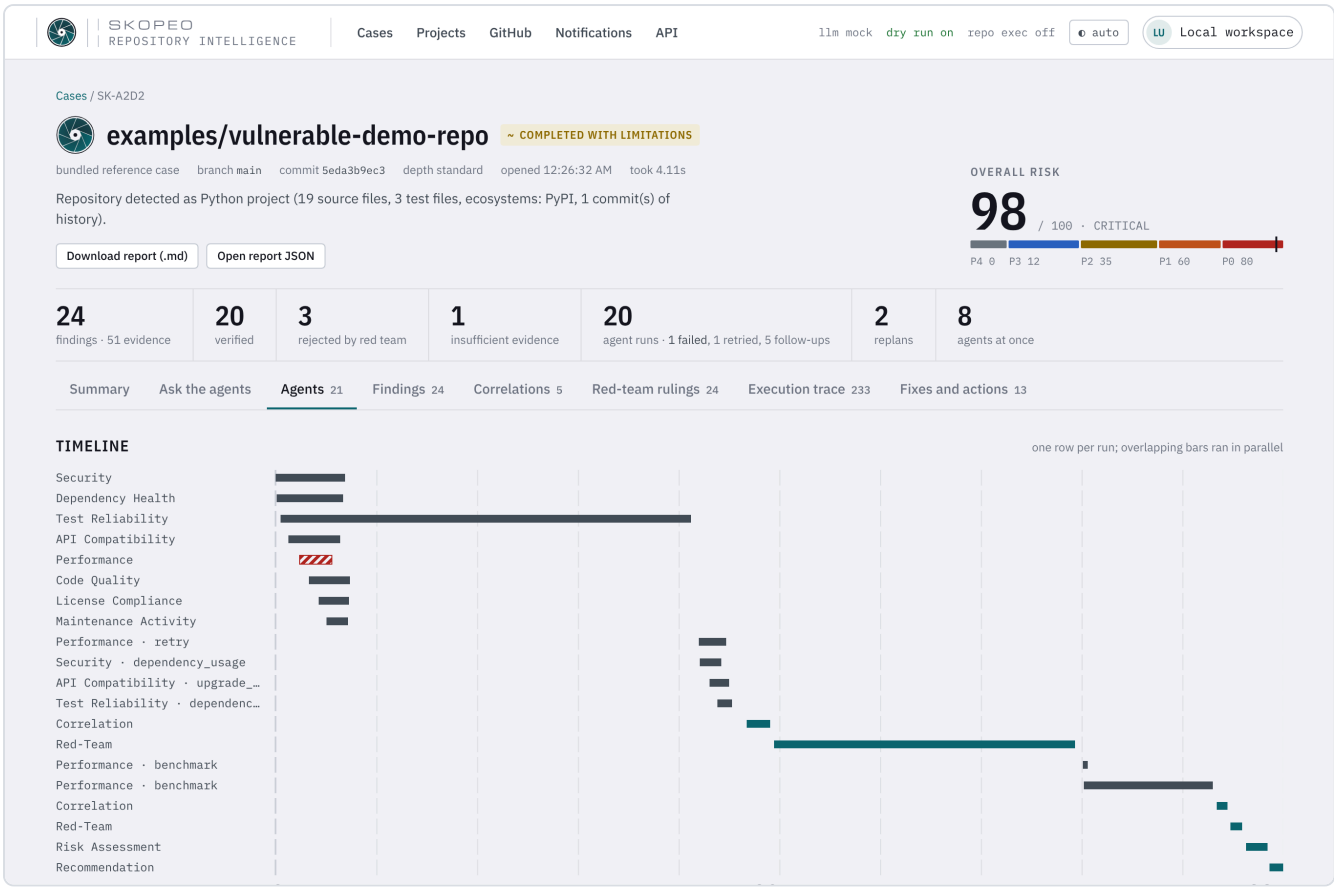


Figure 6. Agent timeline of the traced case: eight specialists in parallel, the failed performance run (hatched) and its retry, follow-ups, correlation, two red-team rounds, benchmarks, risk and recommendation.

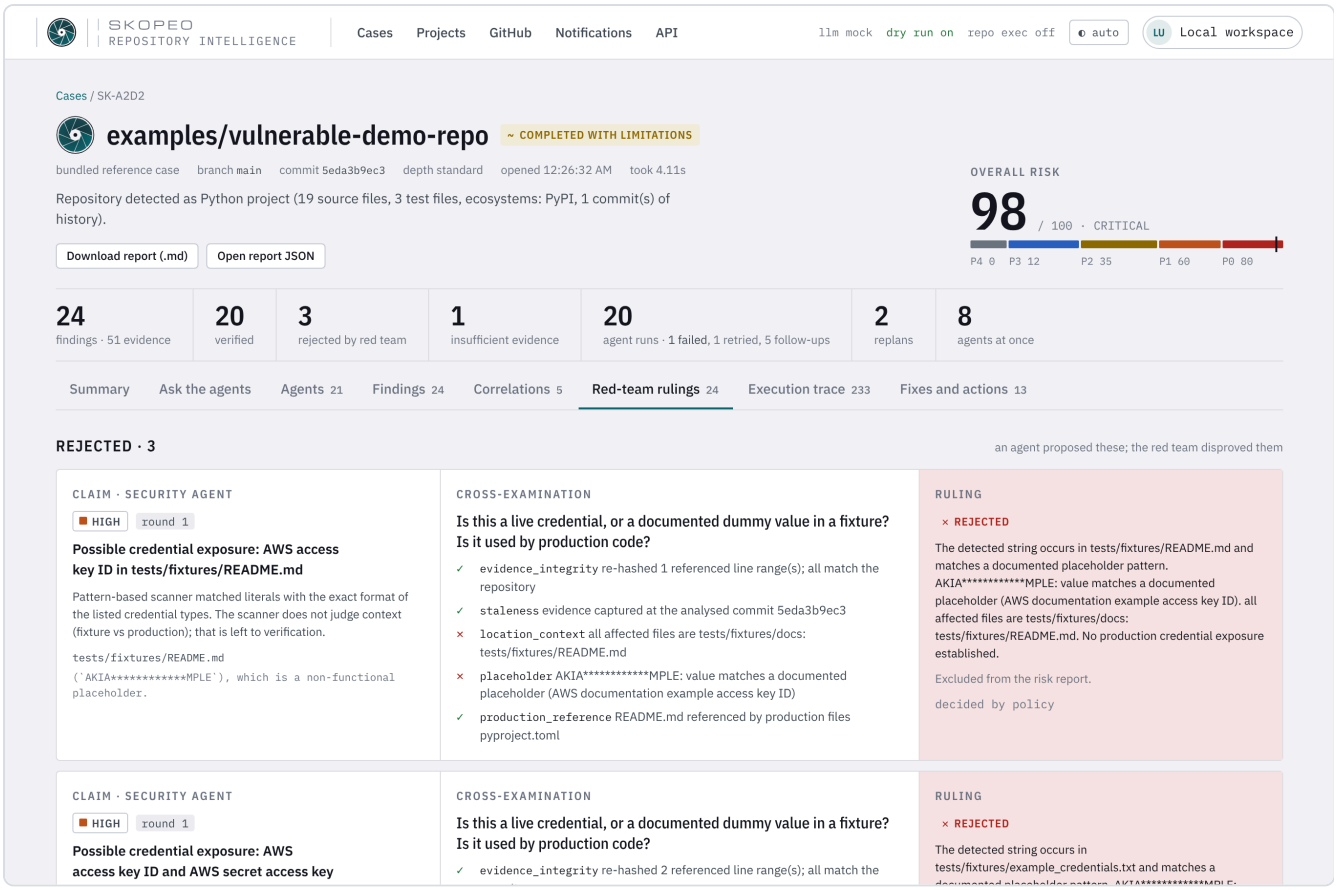


Figure 7. Red-team rulings with the checks behind each decision.

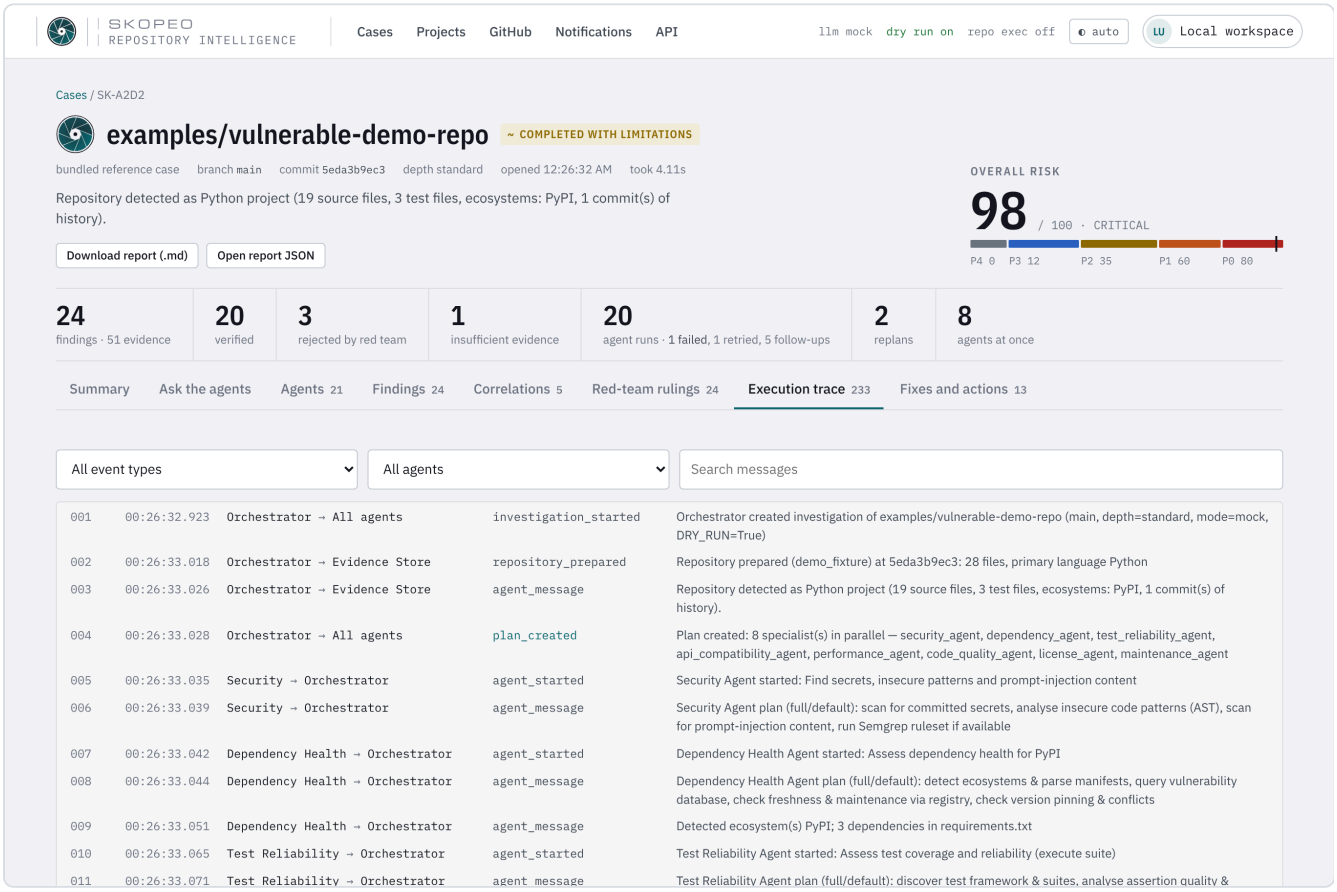


Figure 8. The execution trace view: sender → receiver for every message.